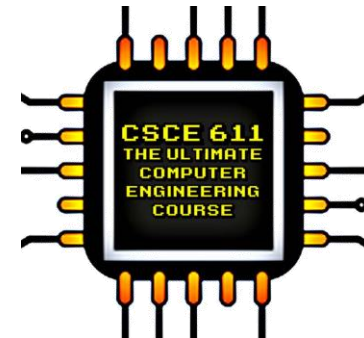


CSCE 611

Advanced Digital Design

Introduction and RISC-V ISA



South Carolina

CSCE 611 Objectives

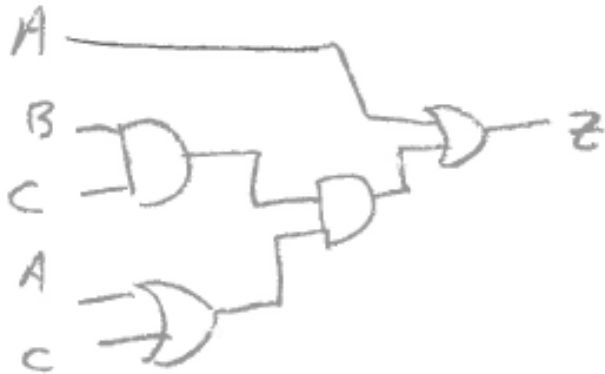
1. Apply CSCE 211 concepts at a larger scale
 - Designs consisting of 1000s of logic gates
2. Design and test a working CPU from scratch

"CSCE 211 Design Flow"

Step 1: Define behavior
(write Boolean equation)

$$Z = A + (B \cdot c) \cdot (A + c)$$

Step 2: "Synthesize" equation into a
gate-level schematic (structure)

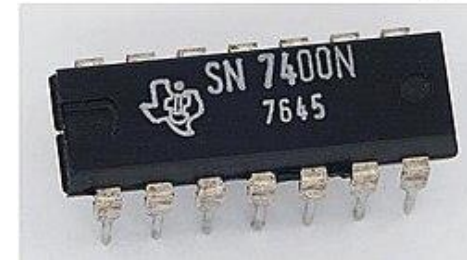
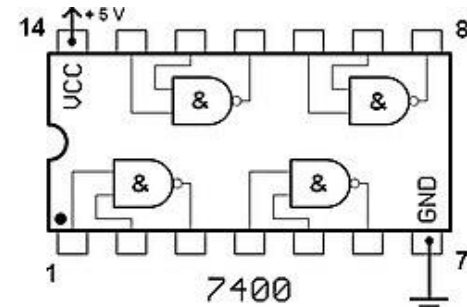


Step 3: "Map" the structure
into fewer, smaller gates

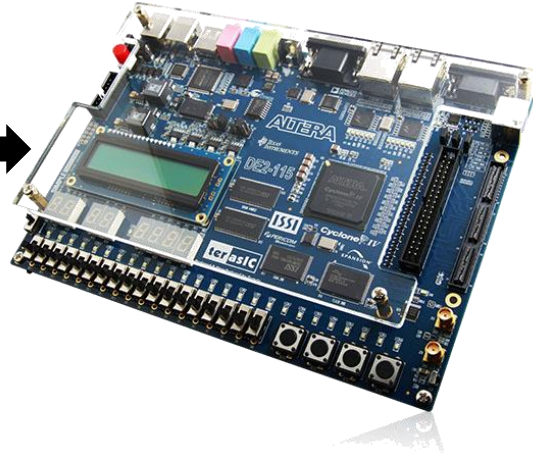
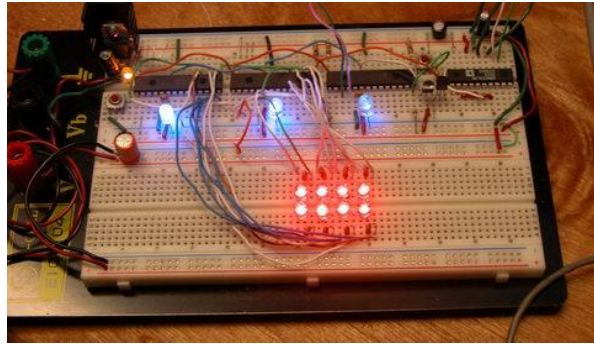
$$\begin{aligned} Z &= A + (ABC + cBc) \\ &= A + (ABC + Bc) \\ &= A + Bc(A + 1) \\ &= A + Bc \end{aligned}$$



Step 4: Place each gate onto
a "slot" on a 7400-series chip
(place-and-route)



CSCE 611 Design Flow



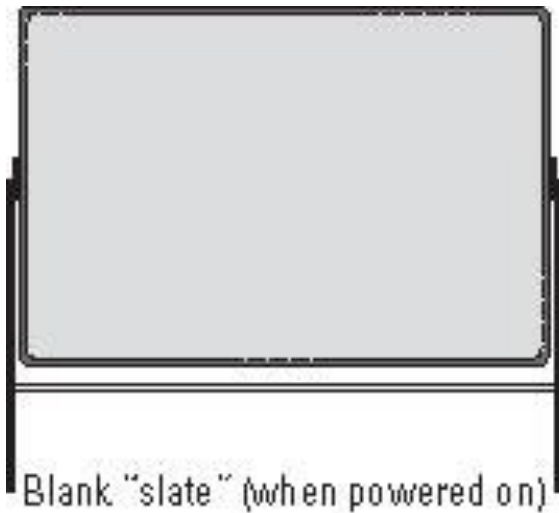
Field Programmable Gate Array (FPGA)

Hardware Description Language (HDL)


$$\begin{aligned} Z &= A + (ABC + CBc) \\ &= A + (ABC + Bc) \\ &= A + Bc(A + 1) \\ &= A + Bc \end{aligned}$$

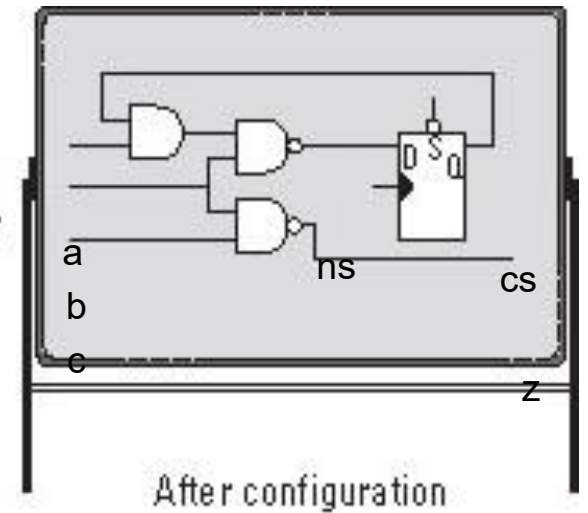

```
module top(input logic clk,a,b,c
           output logic z);
    logic ns,cs;
    assign ns = (cs&a)~&b;
    assign z = b~&c;
    always_ff @(posedge clk)
        cs <= ns;
endmodule
```

FPGA



```
module top(input logic clk,a,b,c
           output logic z);
  logic ns,cs;
  assign ns = (cs&a)~&b;
  assign z = b~&c;
  always_ff @(posedge clk)
    cs <= ns;
endmodule
```

Synthesize,
map,
place-and-route



Build a CPU



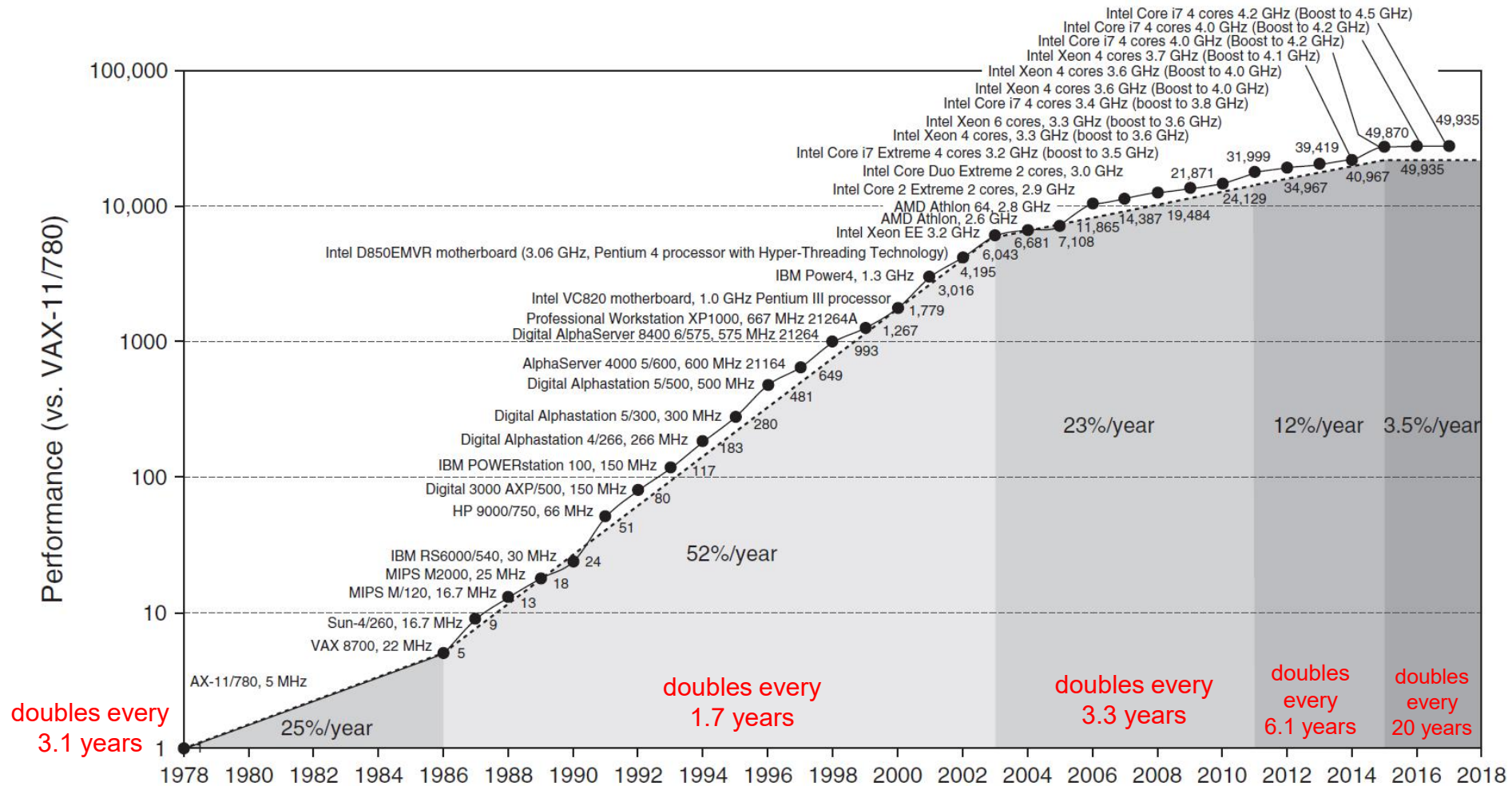
structural HDL (top-down)

```
// Instantiate our control unit  
control_unit my_control_unit(.opcode_i(opcode) ,  
.funct3_i(funct3) ,  
.funct7_i(funct7) ,  
.cssrw_i(cssrw) ,  
.alusrc_o_EX(alusrc_EX) ,  
.gpio_we_o(gpio_we) ,  
.regwrite_o_EX(regwrite_EX) ,  
.regsel_o_EX(regsel_EX) ,  
.aluop_o_EX(aluop_EX) ,  
.stall_o(stall_FCH) ,  
.stall_i(stall_EX) ,  
.pcsrc_o(pcsrch_EX) ,  
.aluoutput_r i(aluoutput r EX));
```

behavioral HDL (bottom-up)

```
// B mux
assign B_EX = alusrc_EX == 1'b0 ?
                                readdata2_EX :
                                immediate_EX;
```

Why Learn Hardware Design?



New Capabilities Enabled by Specialized Hardware

- Tesla Autopilot (2014)
Mobileye EyeQ3



- 4K video on a phone (2015)
Media Engine



iPhone 6 S
The only thing that's changed is everything.
4K
videos

- FaceID (2017)
Secure Enclave



- Animojis (2017)
Apple GPU



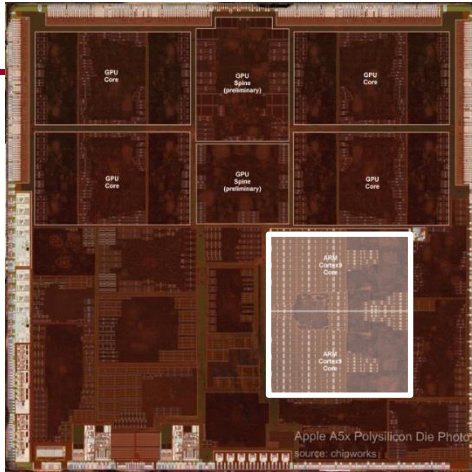
- Computational photography (2018)
Image Signal Processor



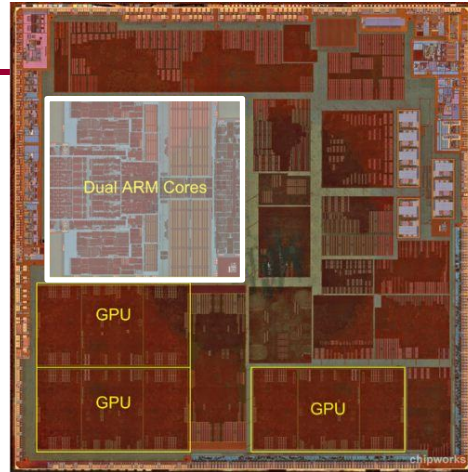
- Live Translation (2025)
Neural Engine



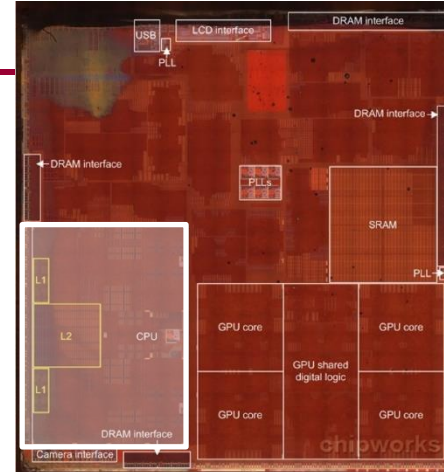
Specialization



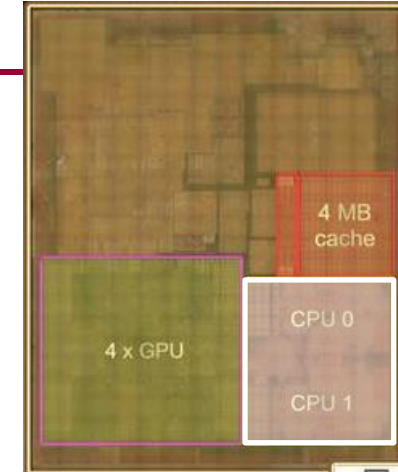
Apple A5x
(2012)



Apple A6
(2012)

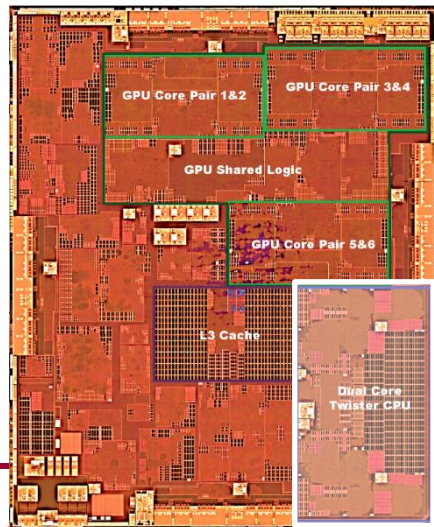


Apple A7
(2013)

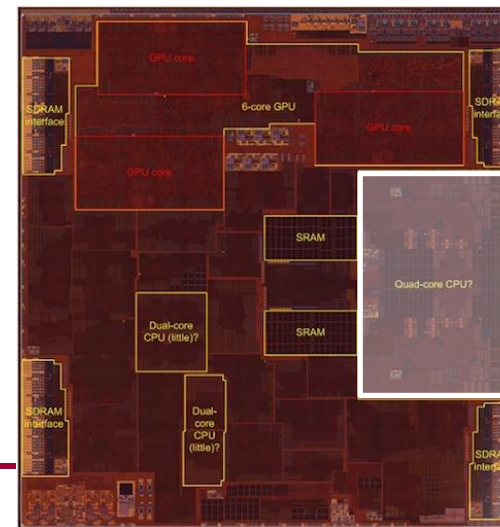


Apple
A8
(2014)

CPU: ~12%



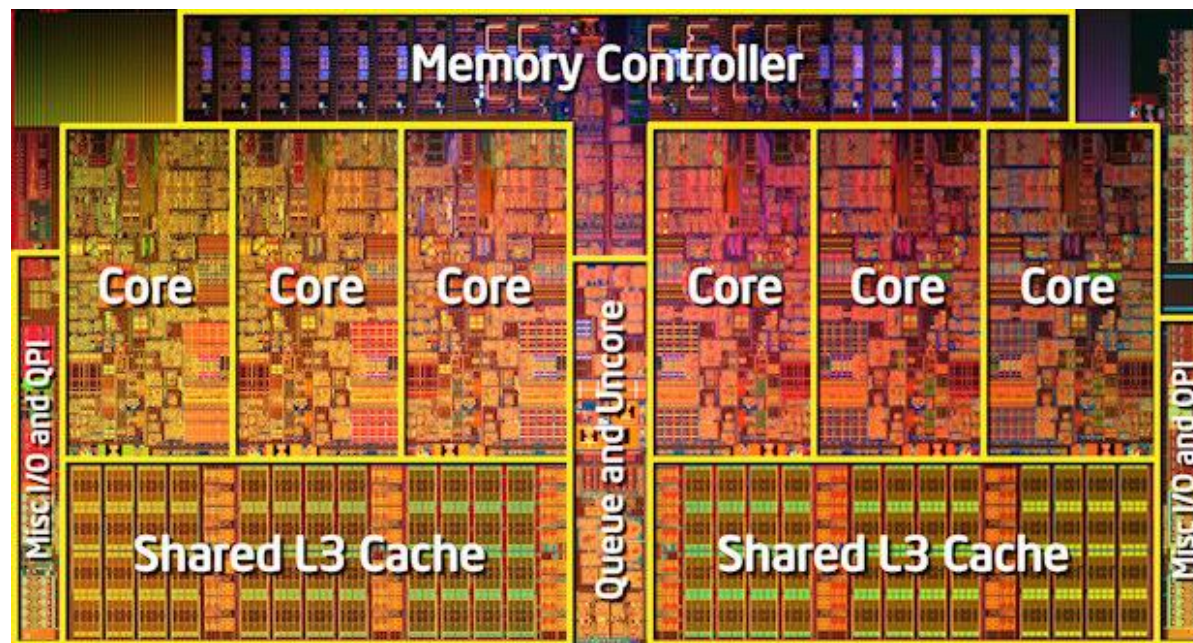
Apple A9
(2015)



Apple A10
(2016)

Performance: Intel Desktop Peak Performance

Desktop Processor	Tech	Year	Max. Clock Speed (GHz)	Integer IPC/core	Cores	Max. DRAM Bandwidth (GB/s)	SP Floating Point (Gflop/s)	L3 cache (MiB)
Westmere i7-620	32	2010	3.33	4	2	17.1	107	4

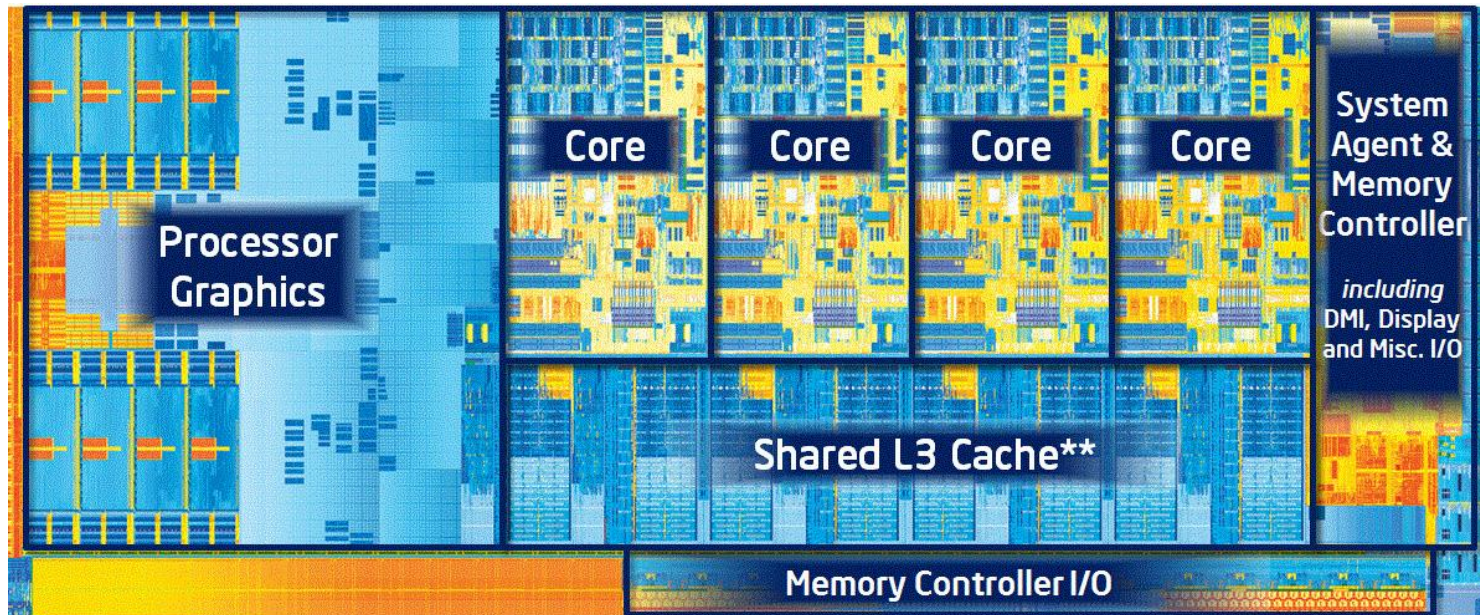


114 mm² die

CPU&cache
= 72%
(82 mm²)

Performance: Intel Desktop Peak Performance

Desktop Processor	Tech	Year	Max. Clock Speed (GHz)	Integer IPC/core	Cores	Max. DRAM Bandwidth (GB/s)	SP Floating Point (Gflop/s)	L3 cache (MiB)
Ivy Bridge i7-3770K	22	2013	3.90	6	4	25.6	250	8

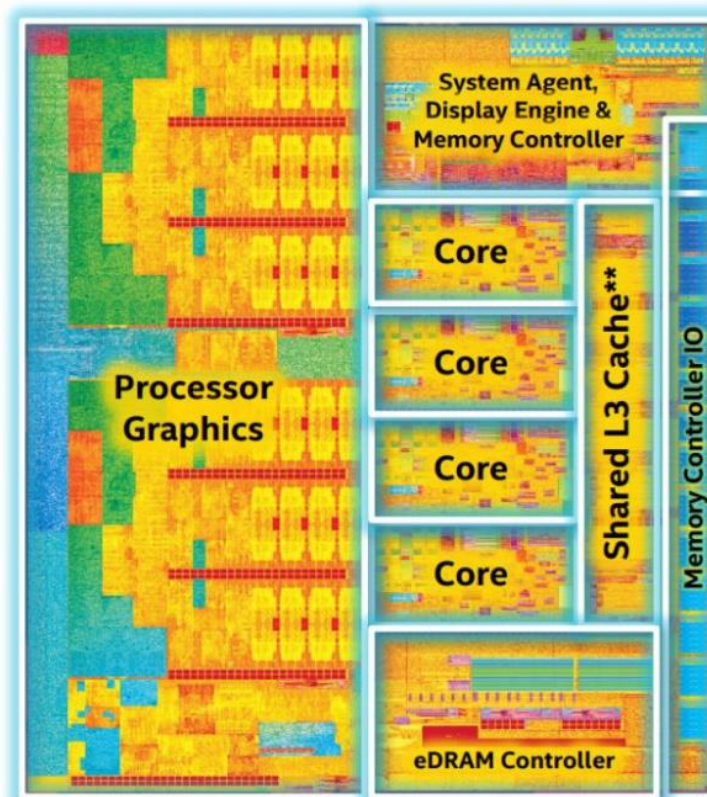


160 mm²
die

CPU &
cache =
47%
(75 mm²)

Performance: Intel Desktop Peak Performance

Desktop Processor	Tech	Year	Max. Clock Speed (GHz)	Integer IPC/core	Cores	Max. DRAM Bandwidth (GB/s)	SP Floating Point (Gflop/s)	L3 cache (MiB)
Broadwell i7-6700K	14	2015	4.20	8	4	34.1	269	8

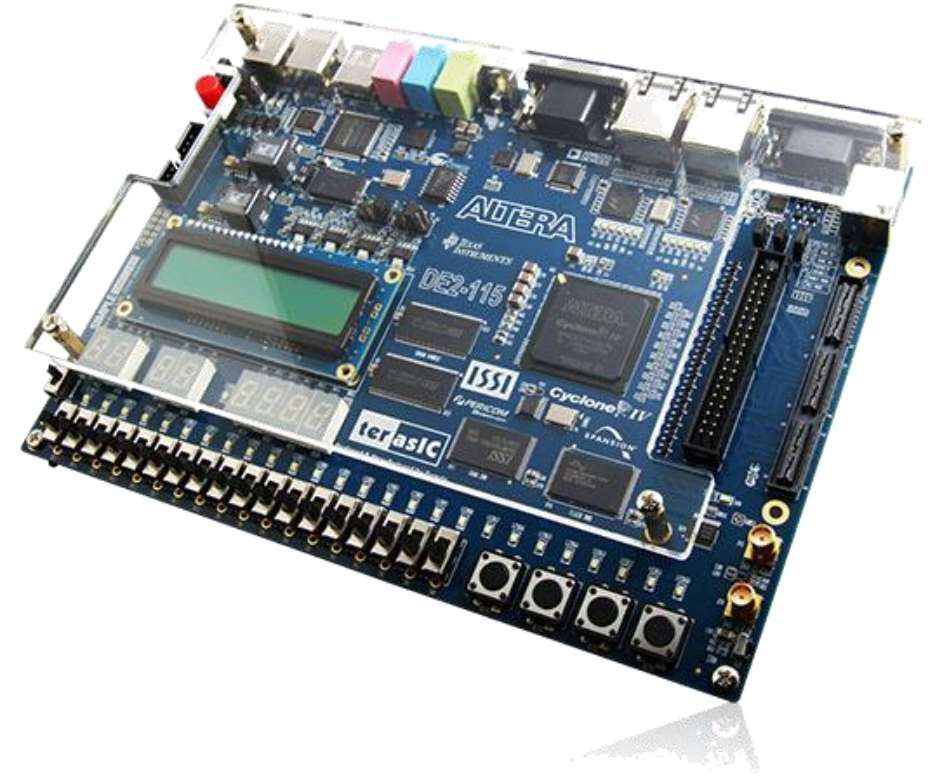


133 mm² die

CPU&cache = 20%
(27 mm²)

This Course

- In lab: design a RISC-V CPU from scratch
- Deploy on FPGA
- Learn for Lab (weeks 1 to 10):
 1. Architecture
 2. Microarchitecture
 3. Large-scale digital design and test
 4. SystemVerilog HDL
- Learn concepts (weeks 11 to 15):
 1. Sequential logic
 2. Digital building blocks
 3. Timing analysis
 4. CMOS logic



DE2 board: Intel Cyclone 4
FPGA with 115K gates

Overview

- In this lecture:
 1. Review basic architecture concepts
 2. Review assembly language programming
 3. RISC-V instruction set
 4. Fixed-point arithmetic
 5. Lab 1

Architecture vs Microarchitecture (CSCE 212)

The "native programming language" of the CPU hardware

Architecture

Assembly language program (for MIPS)

```
swap:
    muli $2, $5, 4
    add $2, $4, $2
    lw $15, 0($2)
    lw $16, 4($2)
    sw $16, 0($2)
    sw $15, 4($2)
    jr $31
```

← without knowing this

Assembler

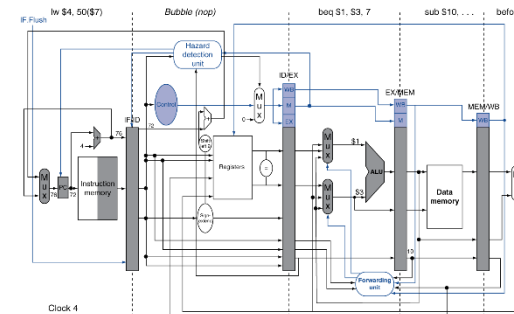
Binary machine language program (for MIPS)

```
000000001010000100000000000011000
000000000000110000001100000100001
10001100011000100000000000000000
10001100111100100000000000000100
10101100111100100000000000000000
10101100011000100000000000000100
0000001111100000000000000001000
```

Software/Hardware Interface

How the ISA is implemented "under the hood"

Microarchitecture



← you can't build this...

Instructions

- From the software prospective, smallest unit of workload for processor
- High level languages need to be translated into instructions:
 - Line of C/C++ code => ~1 to 10 instructions
 - Line of Java code => ~10 to 100 instructions
 - Line of Python code => ~100 to 1000 instructions
- Instructions are:
 1. Identified by their mnemonic
 - Examples: "add", "sub", "jump"
 2. Generally limited to two inputs and one output
 3. Can only do one of three things:
 - 2-input arithmetic or logical operation
 - Bring in one word from memory and save one word to memory
 - Make a decision with regard to where to find the next instruction

RISC-V

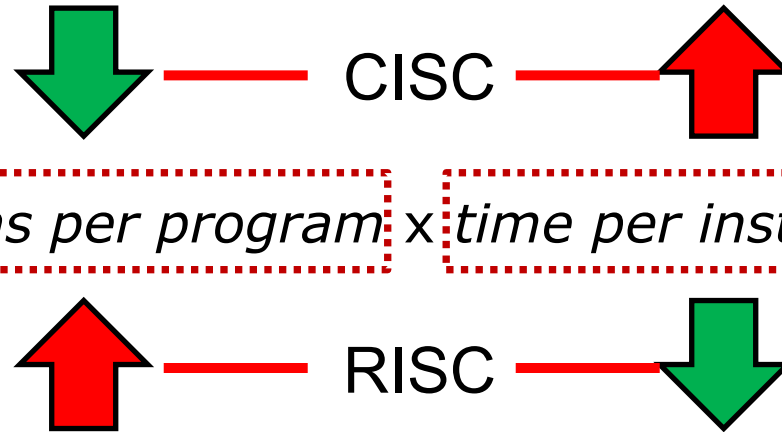
- Similar to MIPS, which you learned in CSCE 212
 - RISC-V fixes MIPS annoyances, e.g.:
 - The destination register address is always in the same spot in the instruction
 - No "shamt" field
 - No zero extension of immediates
 - All jumps generate a return address
 - RISC-V retains some MIPS annoyances, e.g.
 - No index registers
 - No autoincrement base registers
 - Store instructions place destination on right, while all other instructions on left
 - RISC-V adds some complexity
 - Increased number of encoding formats

What is RISC?

- Design “philosophies” for ISAs: RISC vs. CISC
 - CISC = Complex Instruction Set Computer
 - RISC = Reduced Instruction Set Computer

- Tradeoff:

- Execution time = *instructions per program* x *time per instruction*



RISC-V ISA

- RISC-V is modular, containing base and extensions
 - RV32I: base 32-bit instruction (arithmetic, load/store, control flow): 47 instructions
 - RV64I: additional instructions for 64-bit integer support: 14 instructions
 - M: integer multiplication and division: 8 instructions
 - A: atomic memory operations for synchronization: 11 instructions
 - F: single-precision floating-point operations: 25 instructions
 - D: double-precision floating-point operations: 25 instructions
 - Q: quad-precision floating-point operations: 27 instructions
 - C: compressed (16-bit) encodings for code density: 36 instructions
 - V: vector operations for parallel processing: 186 instructions
 - H: hypervisor support for virtualization: 2 instructions
 - Z: a group of "subextensions" and self-contained extensions (e.g. half-precision arithmetic, crypto, new vector operations, vector cryptography, cache prefetch hints, etc.)

Outline for Today

- Welcome to our first official lecture!
- Agenda:
 1. Review last lecture
 2. Finish introducing RISC-V ISA
 3. Run some example code in RARS
 4. Fixed point representation and arithmetic
 5. Introduce Lab 1

Registers

- RISC-V (like MIPS) has 32 general purpose registers
 - In machine language: referenced as a 5-bit number
 - In assembly language: referenced as x0 to x31
 - Recall: MIPS assembly used \$0 to \$31
- Like MIPS, RISC-V registers have alternative names:
 - s0 to s11: (supposed to be) preserved across function calls
 - t0 to t6: not preserved across function calls
 - a0 to a7: function arguments and return values
 - ra, sp, gp, tp
 - zero

Register Map (Physical -> "Symbolic")

Register	Symbolic name
x0	zero
x1	ra
x2	sp
x3	gp
x4	tp
x5 to x7	t0 to t2
x8 to x9	s0 to s1
x10 to x17	a0 to a7
x18 to x27	s2 to s11
x28 to x31	t3 to t6

The Constant Zero

- RISC-V register 0 (zero) is the constant 0
 - Cannot be overwritten
- Useful for common operations
 - e.g., move between registers
add t2, s1, zero

Immediate Operands

- Constant data specified in an instruction

`addi s3, s3, 4`

- No subtract immediate instruction

- Use a negative constant

`addi s2, s1, -1`

Assembly Language Challenges

Having to load and store separately from compute and allocating registers

$f = (g + h) - (i + j);$

WRONG

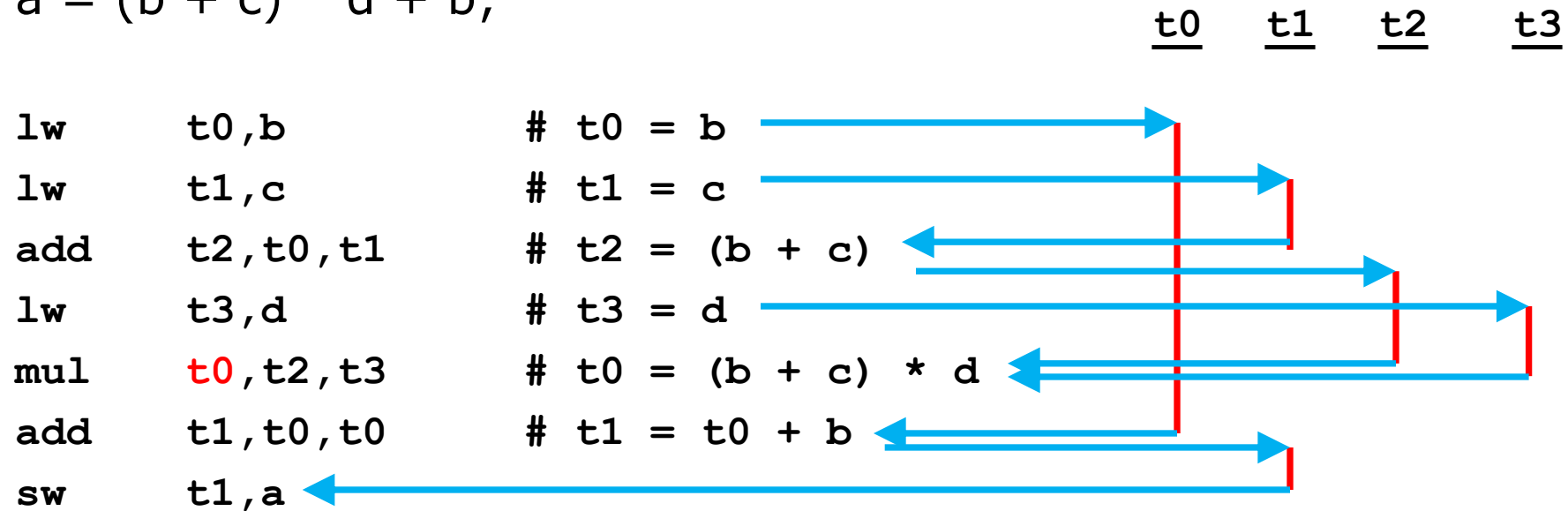
```
add temp1, g, h
add temp2, i, j
sub f, temp1, temp2
```

CORRECT

```
lw s1,g
lw s2,h
lw s3,i
lw s4,j
add t0, s1, s2 # g+h
add t1, s3, s4 # i+j
sub s0, t0, t1
sw s0,f
```

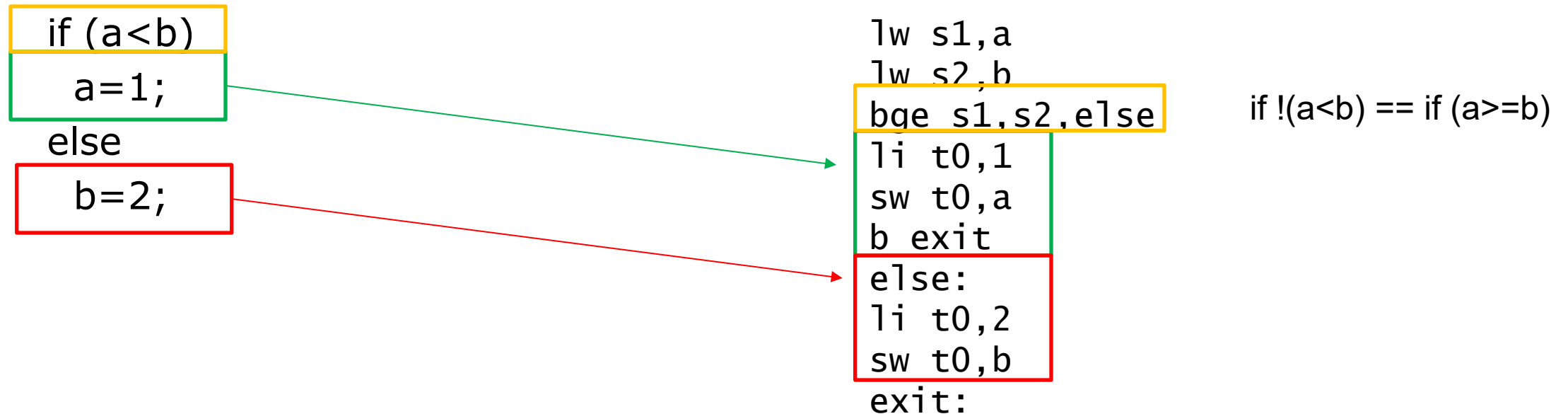
Assembly Programming Challenges

$a = (b + c) * d + b;$



Assembly Language Challenges

if-statement logic must be **complemented** when using conditional branches



Assembly Programming Challenges

No index for array access:

```
int vals[100];  
for (int i = 0; i < 100; i++) sum += vals[i];
```

- There is no hardware support for indices in RISC-V

WRONG

```
li x1,0  
li x4,100  
loop: lw x3,vals[x1]  
      addi x1,x1,1  
      blt x1,x4,loop
```

CORRECT

```
li x1,0  
la x2,vals  
li x4,100  
loop: lw x3,0(x2)  
      addi x2,x2,4  
      addi x1,x1,1  
      blt x1,x4,loop
```

RISC-V Assembly Instruction Types

1. arithmetic, logical, shift, comparison: 3-register

`<mnemonic> <dest>, <src1>, <src2>`

Examples:

```
add x1,x2,x3      # R[x1] <- R[x2] + R[x3]
sll x1,x2,x3      # R[x1] <- R[x2] << R[x3]
slt x1,x2,x3      # R[x1] <- (int32_t)R[x2] < (int32_t)R[x3] ? 1 : 0
sltu x1,x2,x3     # R[x1] <- (uint32_t)R[x2] < (uint32_t)R[x3] ? 1 : 0
and x1,x2,x3      # R[x1] <- R[x2] & R[x3]
mul x1,x2,x3      # R[x1] <- (int32_t)R[x2] * (int32_t)R[x3]
mulh x1,x2,x3     # R[x1] <- ((int32_t)R[x2] * (int32_t)R[x3]) >> 32
mulhu x1,x2,x3    # R[x1] <- ((uint32_t)R[x2] * (uint32_t)R[x3]) >> 32
```

RISC-V Assembly Instruction Types

2. arithmetic, logical, shift, comparison: 2-register+immediate

`<mnemonic> <dest>,<src1>,<immediate>`

Examples:

```
addi x1,x2,10      # R[x1] <- R[x2] + 10
andi x1,x2,10      # R[x1] <- R[x2] & 0x1010
slli x1,x2,10      # R[x1] <- R[x2] << 10
```

RISC-V Assembly Instruction Types

3. Branch

`<mnemonic> <src1>,<src2>,<label>`

4. Jump

`<mnemonic> <label>`

Examples:

`jal subr # unconditionally go to "subr" label and R[x1] <- return address`

Examples:

`back: blt x1,x2,skip # go to the "skip" label if R[x1] < R[x2]`

`reset: add x1,x0,x2`

`jr x3`

`skip: jal x3,reset`

`nop`

Assembly Programming Challenges

Can't compare to a constant using only one instruction

if (a < 0) return;

WRONG

```
blt x1,10,exit
```

CORRECT

```
li x2,10  
blt x1,x2,exit
```

RISC-V Assembly Instruction Types

5. load/store

`<mnemonic> <reg>,label`

`<mnemonic> <reg>,<offset>(<base>)`

Examples:

```
lw x1,48(x2)      # R[x1] <- Mem[R[x2]+48]
```

Access `foo[i]` assuming:

- `foo` declared as `int []`
- `i` is live in `x1`
- Address of "`foo`" is live in `x2`

```
slli x3,x1,2      # multiply i by 4
```

```
add x3,x2,x3      # add index offset to R[x2]
```

```
lw x3,0(x3)      # load the word
```

RISC-V Assembly Instruction Types

6. "coprocessor" instructions

`<mnemonic> <reg1>,<imm>,<reg2>`

`csrrw rd,imm,rs1   # R[rd] <- coproc[imm]; coproc[imm] <- R[rs1]`

Examples:

`csrrw x2,0xf00,x3`

RISC-V Assembly Instruction Types

1. arithmetic, logical, shift, comparison: 3-register

<mnemonic> <dest>,<src1>,<src2>

add x1,x2,x3

2. arithmetic, logical, shift, comparison: 2-register+immediate

<mnemonic> <dest>,<src1>,<imm>

addi x1,x2,10

3. branch

<mnemonic> <src1>,<src2>,<label>

blt x1,x2,loop

4. jump

<mnemonic> <label>

jal subr

5. load/store

<mnemonic> <reg>,label

<mnemonic> <reg>,<offset>(<base>)

lw x1,48(x2)

6. "coprocessor" instructions

<mnemonic> <reg1>,<imm>,<reg2>

csrrw rd,imm,rs1

Console I/O

```
.data
prmt: .asciz    "Enter a number: "
newln: .asciz   "\n"

.text
main: li    a7,4      # print string
      la    a0,prmt
      ecall
      li    a7,5      # read integer
      ecall
      li    a7,1      # print integer that was entered
      ecall
```

RISC-V Instructions to Use in Lab 1 and Implement in Labs 3 and 4

Basic arithmetic and bitwise logical:

<code>add rd, rs1, rs2</code>	<code># R[rd] = R[rs1] + R[rs2]</code>
<code>sub rd, rs1, rs2</code>	<code># R[rd] = R[rs1] - R[rs2]</code>
<code>and rd, rs1, rs2</code>	<code># R[rd] = R[rs1] & R[rs2]</code>
<code>or rd, rs1, rs2</code>	<code># R[rd] = R[rs1] R[rs2]</code>
<code>xor rd, rs1, rs2</code>	<code># R[rd] = R[rs1] ^ R[rs2]</code>

Shift instructions:

<code>sll rd, rs1, rs2</code>	<code># R[rd] = R[rs1] << R[rs2]</code>
<code>sra rd, rs1, rs2</code>	<code># R[rd] = (signed)R[rs1] >> R[rs2]</code>
<code>srl rd, rs1, rs2</code>	<code># R[rd] = (unsigned)R[rs1] >> R[rs2]</code>

Comparison instructions:

<code>slt rd, rs1, rs2</code>	<code># R[rd] = (signed)R[rs1] < (signed)R[rs2]</code>
<code>sltu rd, rs1, rs2</code>	<code># R[rd] = (unsigned)R[rs1] < (unsigned)R[rs2]</code>

RISC-V Instructions to Use in Lab 1 and Implement in Labs 3 and 4

Multiply instructions:

`mul rd, rs1, rs2`

`# R[rd] = R[rs1] * R[rs2]`

`mulh rd, rs1, rs2`

`# R[rd] = ((signed)R[rs1] * (signed)R[rs2]) >> 32`

`mulhu rd, rs1, rs2`

`# R[rd] = ((unsigned)R[rs1] * (unsigned)R[rs2] ) >> 32`

RISC-V Instructions to Use in Lab 1 and Implement in Labs 3 and 4

Basic arithmetic and bitwise logical:

```
addi rd, rs1, immediate  # R[rd] = R[rs1] + immediate
andi rd, rs1, immediate  # R[rd] = R[rs1] & immediate
ori  rd, rs1, immediate  # R[rd] = R[rs1] | immediate
xori rd, rs1, immediate  # R[rd] = R[rs1] ^ immediate
```

Shift instructions:

```
slli rd, rs1, immediate  # R[rd] = R[rs1] << immediate
srai rd, rs1, immediate  # R[rd] = (signed)R[rs1] >> immediate
srli rd, rs1, immediate  # R[rd] = (unsigned)R[rs1] >> immediate
```

RISC-V Instructions to Use in Lab 1 and Implement in Labs 3 and 4

Branch and Jump Instructions:

```
beq rs1, rs2, <label> # if R[rs1] == R[rs2] then branch to target
bne rs1, rs2, <label> # if R[rs1] != R[rs2] then branch to target
bge rs1, rs2, <label> # if (signed)R[rs1] > (signed)R[rs2] then branch to target
bgeu rs1, rs2, <label> # if (signed)R[rs1] >= (signed)R[rs2] then branch to target
blt rs1, rs2, <label> # if (signed)R[rs1] < (signed)R[rs2] then branch to target
bltu rs1, rs2, <label> # if (unsigned)R[rs1] < (unsigned)R[rs2] then branch to target

b <label>                # unconditionally branch to target (resolve to jal instruction)
jal rd, <label>          # R[rd] = return address (PC+4), jump to label
jalr rd, rs, imm         # R[rd] = return address (PC+4), PC = R[rs] + imm
```

RISC-V Instructions to Use in Lab 1 and Implement in Labs 3 and 4

Load and Store Instructions (not included in course CPU):

```
lw rd, <label>          # R[rd] = Mem[label]
sw rs2, <label>          # Mem[label] = R[rs2]
```

Load Upper Immediate:

```
lui rd,imm (U-type)      # R[rd] = imm(20 bit) << 12
```

Fixed Point

- How do you convert binary to decimal?

- Answer:

- $bit_0 \times 2^0 + bit_1 \times 2^1 + bit_2 \times 2^2 + \dots$

- Example: 1010011

1	0	1	0	0	1	1
<i>bit₆</i>	<i>bit₅</i>	<i>bit₄</i>	<i>bit₃</i>	<i>bit₂</i>	<i>bit₁</i>	<i>bit₀</i>

- $1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$
- $= 64 + 16 + 2 + 1$
- $= 83$

Fixed Point

- Maximum value (unsigned):

1	1	1	1	1	1	1
<i>bit₆</i>	<i>bit₅</i>	<i>bit₄</i>	<i>bit₃</i>	<i>bit₂</i>	<i>bit₁</i>	<i>bit₀</i>

- $2^7 - 1 = 127$
- Range = $[0, 127]$

- Maximum value (signed):

1	0	0	0	0	0	0
<i>sign</i>	<i>bit₅</i>	<i>bit₄</i>	<i>bit₃</i>	<i>bit₂</i>	<i>bit₁</i>	<i>bit₀</i>

- $-2^6 = -64$
- Range = $[-64, 63]$

Fixed Point

- Increment (unit in the last place):

1	1	1	1	1	1	1
<i>sign/bit₆</i>	<i>bit₅</i>	<i>bit₄</i>	<i>bit₃</i>	<i>bit₂</i>	<i>bit₁</i>	<i>bit₀</i>

- $2^0 = 1$

Fixed Point

- What if:

1	0	1	0	0	1	1
<i>bit₄</i>	<i>bit₃</i>	<i>bit₂</i>	<i>bit₁</i>	<i>bit₀</i>	<i>bit₋₁</i>	<i>bit₋₂</i>

- Viewed normally, this is 83
- But now... it is scaled by $2^{-2} = \frac{1}{4}$
- New value = $\frac{83}{4} = 20.75$

Fixed Point

- What if:

1	0	1	0	0	1	1
<i>bit₄</i>	<i>bit₃</i>	<i>bit₂</i>	<i>bit₁</i>	<i>bit₀</i>	<i>bit₋₁</i>	<i>bit₋₂</i>
Whole part					Fractional part	

- This is a $(m,n) = (7,2)$ value
- Increment = $2^{-2} = 0.25$
- Range (unsigned) = $\left[0, \frac{2^m}{2^n} - 2^{-n}\right]$
- Range (signed) = $\left[-\frac{2^{m-1}}{2^n}, \frac{2^{m-1}}{2^n} - 2^{-n}\right]$

Fixed Point

- Fixed-point
 - Assume a decimal point at some location in a value:

1	0	1	1	0	1
---	---	---	---	---	---

As 6-bit integer: $2^5 + 2^3 + 2^2 + 2^0 = 32 + 8 + 4 + 1 = 45$

1	0	1.	1	0	1
---	---	----	---	---	---

As (6,3)-fixed format: $2^2 + 2^0 + 2^{-1} + 2^{-3} = 5 + .5 + .125 = 5.625$
 $= 5 + 5/8$

1	0.	1	1	0	1
---	----	---	---	---	---

As (6,4)-fixed format: $2 + 13/16 = 2.8125$

1	0.	1	1	0	1
---	----	---	---	---	---

45

2.8125

x

1	0.	1	1	0	1
---	----	---	---	---	---

45

2.8125

=

0	1	1	1.	1	1	1	0	1	0	0	1
---	---	---	----	---	---	---	---	---	---	---	---

2025

$7 + (128+64+32+8+1)/256$

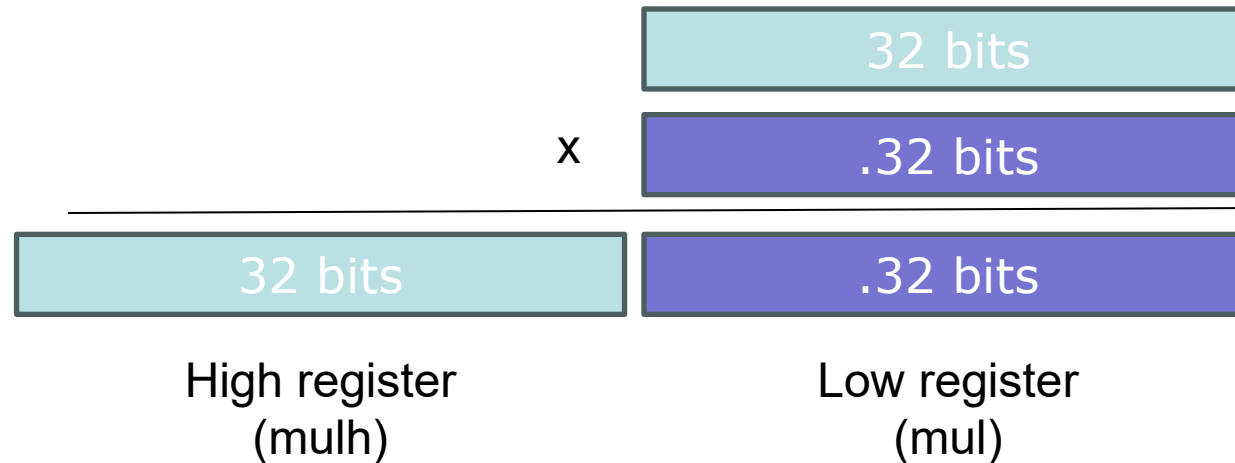
$7 + 233/256 = 7.91015625$

Fixed Point Range

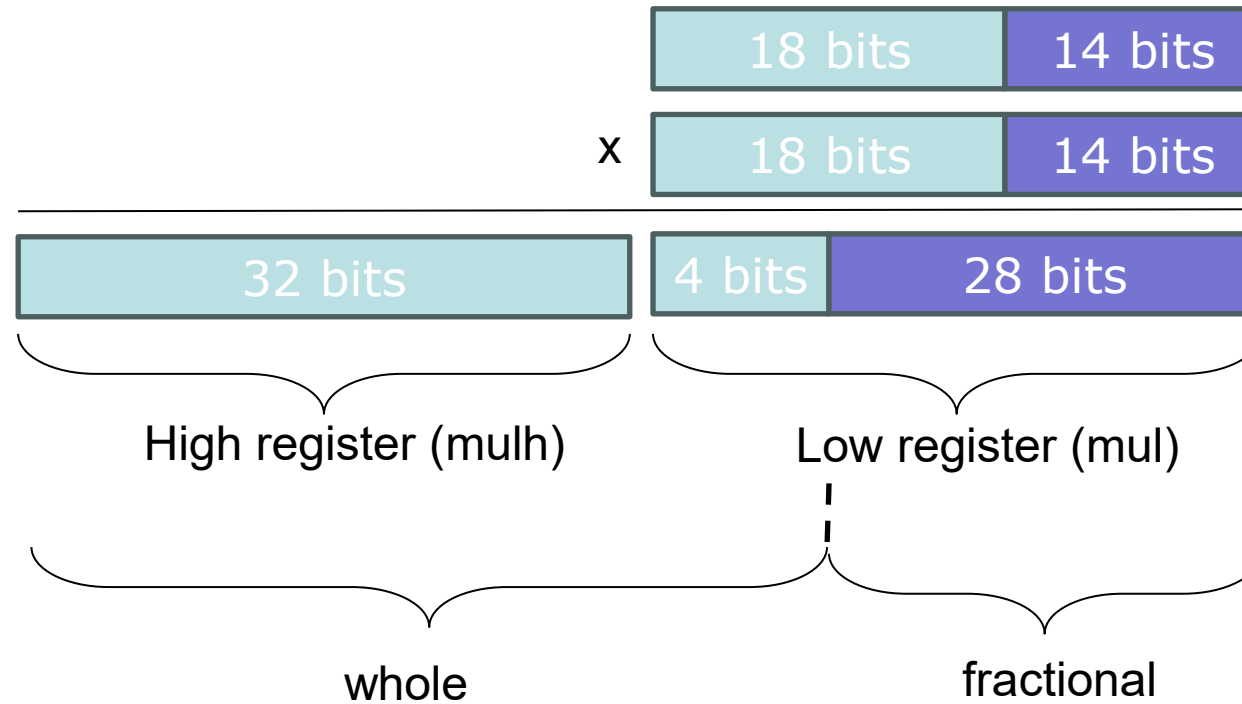
- Range depends on total number of whole and fractional bits
- Precision depends on number of leading 0s
- Note:
 - Floating point range depends on number of exponent bits
 - Precision depends on number of mantissa/significand bits

Fixed Point Multiply

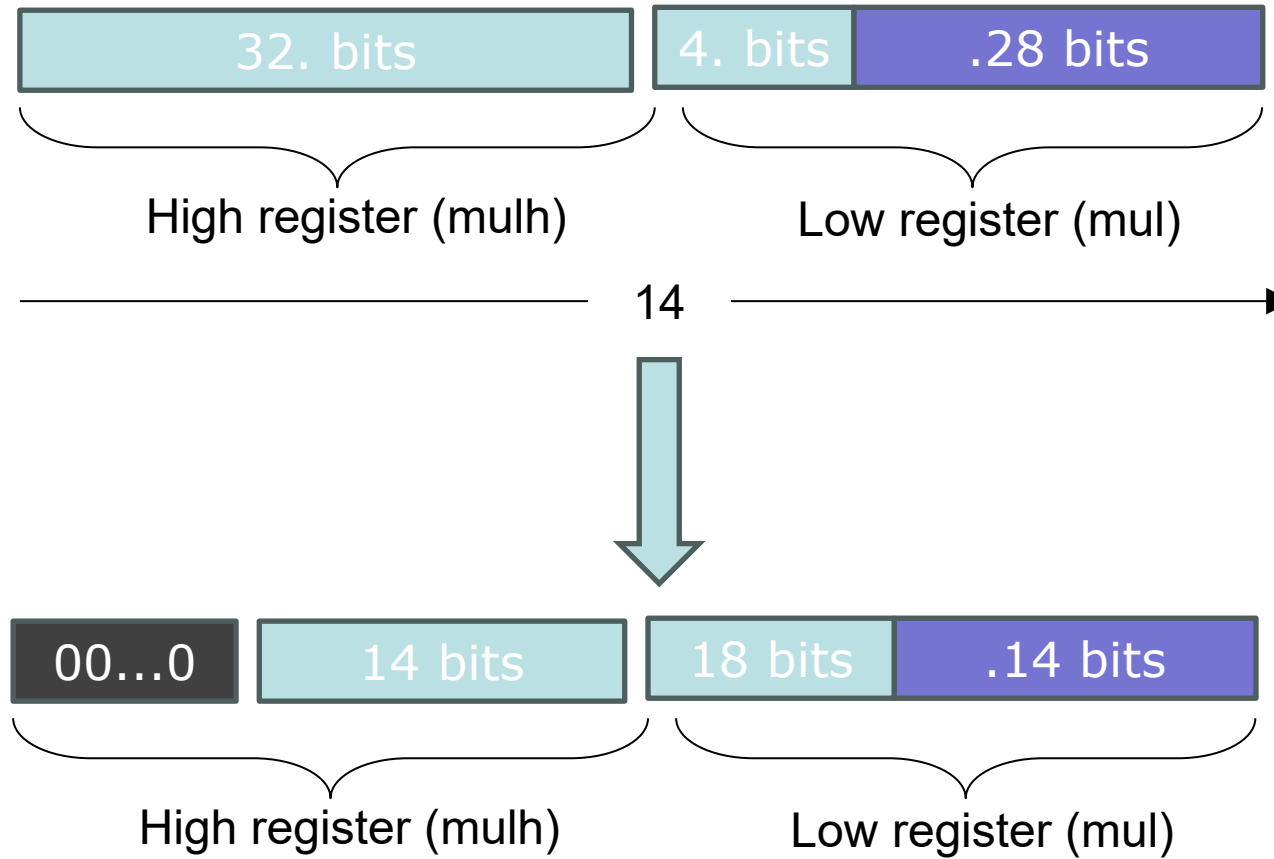
- Multiplying values of types (m_1, n_1) and (m_2, n_2) will produce value of type $(m_1 + m_2, n_1 + n_2)$
- Example:
 - $(32, 0)$ value (no fractional part) multiplied by $(32, 32)$ (all fractional bits):



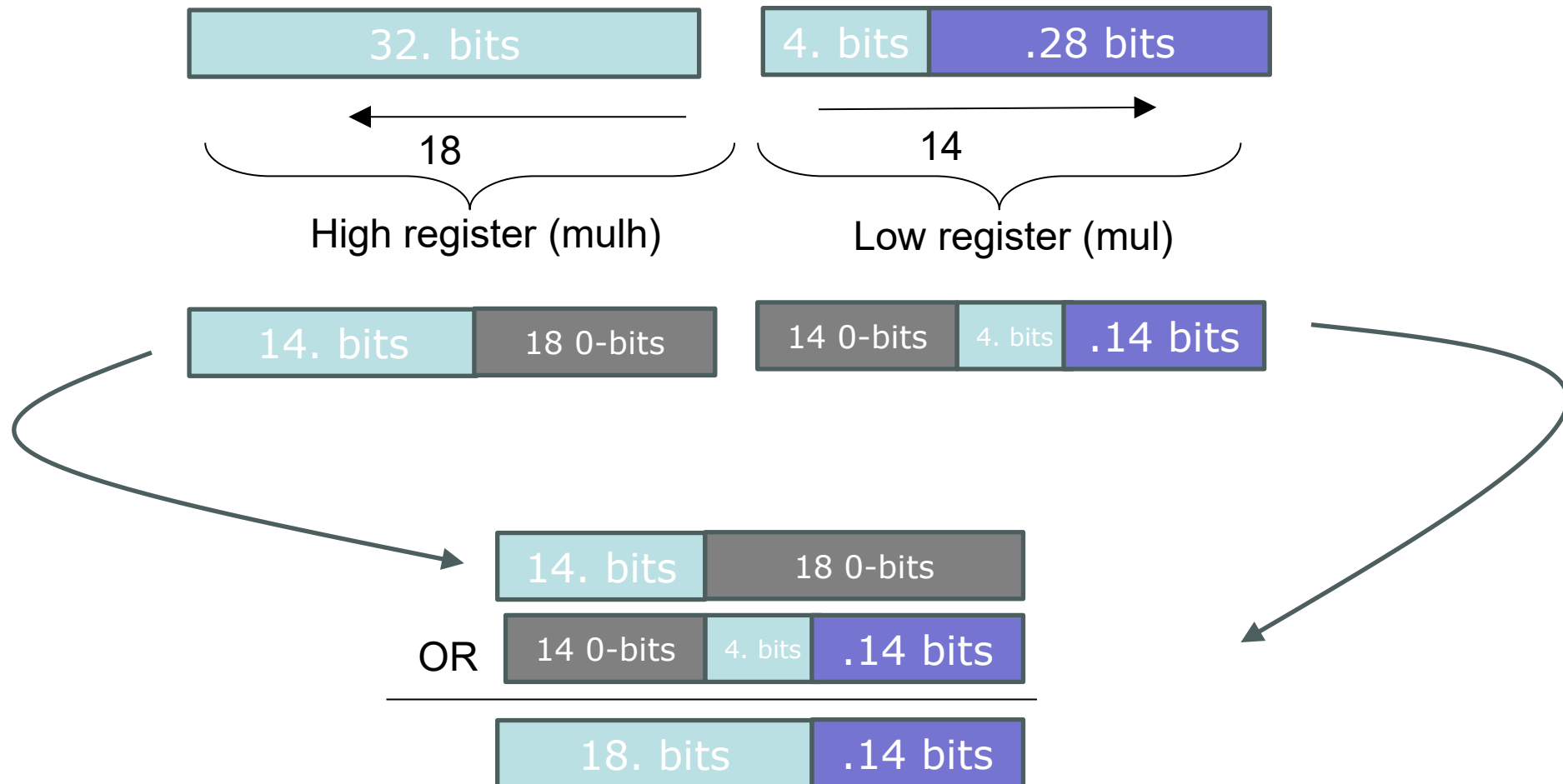
Fixed Point Multiply



Shift Result



Shift Result



Square Root

- Basic idea:
 - For input value **S**
 - Write a problem that solves equation $x^2 - S = 0$
 - Simplest solution: use binary search
 1. Choose initial guess for $X = 0.0$
 2. Choose initial **step**
 3. Repeat until $\text{step} == 0$:
 1. $\text{val} = x^2 - S$
 2. if $x^2 < S$ then $x = x + \text{step}$ else $x = x - \text{step}$
 3. $\text{step} = \text{step} / 2$

- Assume $S=2$

x	x ²	action	step
0	0	add	2
2	4	sub	1
1	1	add	.5
1.5	2.25	sub	.25
1.25	1.5625	add	.125
1.375	1.8906	add	.0625
1.4375	2.0664	sub	.03125
1.40625	1.9775	sub	0.01563
...			

Lab 1

- Assume the "actual" input is a (32,14) input value but given as a (32,0) "raw" number:
 - Example: enter 45, interpreted as $\frac{45}{2^{14}} = 0.00274658203125$
- Compute the square root as a (32,14) value
 - Example: $\sqrt{\frac{45}{2^{14}}} = \pm 0.05240784322265132100959000786089$
- Keep positive result
- Round down to nearest bit
 - Example: $[0.05240784322265132100959000786089 \times 2^{14}] = 859$
- Print result

RARS Workaround

- RARS only accepts signed integers



- To access input values from 2^{31} to $2^{32} - 1$:

$$-(2^{32} - \text{value})$$

Example: a signed 4-bit number has range -8 to 7

If we want 10, then enter $-(2^4 - 10) = -6$

-6 is 1010

s	b	u
-8	1000	8
-7	1001	9
-6	1010	10
-5	1011	11
-4	1100	12
-3	1101	13
-2	1110	14
-1	1111	15
0	0000	0
1	0001	1